CloudGuard Kubernetes Sandbox

This setup creates a dedicated single-node k3s cluster for CloudGuard's intentionally vulnerable live demo. Never use a production cluster or connect this VM to production networks or data.

Required VM

- Ubuntu 22.04 or 24.04
- 2 vCPU, 4 GB RAM, and 40-60 GB disk
- A stable private IP or DNS name reachable from the Dokploy VM
- Outbound HTTPS access for the k3s installer and container images
- Inbound TCP 22 from administrator addresses only
- Inbound TCP 6443 from the Dokploy VM only
- No public access to TCP 80, 443, or 30000-32767

Traefik and ServiceLB are disabled. This prevents the lab's LoadBalancer service and K3s defaults from claiming the VM's web ports. The NodePort finding remains visible through the Kubernetes API without exposing the NodePort publicly.

Install

Copy this directory to the dedicated VM, then run:

```
cd deploy/kubernetes-lab
chmod 700 install-k3s-lab.sh issue-cloudguard-kubeconfig.sh
sudo K3S_API_HOST=<private-ip-or-dns> ./install-k3s-lab.sh
sudo K3S_API_HOST=<private-ip-or-dns> ./issue-cloudguard-kubeconfig.sh
```

The installer grants `cluster-admin` only to the dedicated `cloudguard-lab-worker` service account. The current demo needs to create and delete temporary namespaces and deliberately create unsafe RBAC for scanner validation. This permission is acceptable only because the cluster is disposable and isolated.

Connect CloudGuard

1. Securely retrieve `/root/cloudguard-kubeconfig.yaml` without printing it into logs or chat.
2. Sign in to CloudGuard and open **Manage Credentials**.
3. Add a Kubernetes credential named `CloudGuard Kubernetes Sandbox`.
4. Paste the kubeconfig, mark it as default, and validate it.
5. Set `SANDBOX_KUBERNETES_DEPLOY=true` in the CloudGuard Dokploy Compose environment.
6. Keep `SANDBOX_LAB_INLINE_WORKER=false` so the separate sandbox worker performs the work.
7. Redeploy CloudGuard.

Acceptance Test

1. Open the vulnerable-lab catalog and confirm Kubernetes reports both `deployment_flag_enabled` and `credential_ready` as true.
2. Start **Deploy and Scan Vulnerable Kubernetes**.
3. Confirm a temporary `cg-*` namespace appears.
4. Confirm the scan produces Kubernetes findings and GRC evidence.
5. Confirm the namespace is deleted after scan completion or TTL expiry.
6. On the lab VM, `k3s kubectl get namespaces` must show no leftover `cg-*` namespace.

The generated token defaults to 30 days. Run `issue-cloudguard-kubeconfig.sh` again and replace the stored credential before it expires.

Hostinger Verification (2026-10-01)

The dedicated Hostinger VM has Ubuntu 26.04.1, 1 vCPU, 4 GB RAM, and a 48 GB disk. K3s `v1.36.4+k3s1` was installed and its node and system pods became ready. This is a small demonstration setup, below the recommended 2-vCPU baseline; run only one lab at a time.

`configure-lab-firewall.sh` restricts SSH to administrator addresses and TCP 6443 to the CloudGuard worker address. IPv4 and IPv6 raw firewall rules block public web, NodePort, and overlay traffic before Kubernetes NAT. `cloudguard-lab-firewall.service` reapplies these restrictions before K3s at boot. Update the source addresses before reusing the configuration on another VM.

The SSH-tunnel smoke test created the actual eight CloudGuard lab objects, started the demo pod, and found 14 lab-specific findings with no Kubernetes API errors. It verified namespace deletion after scanning. This tests the local scanner against the remote cluster; it does not prove the live portal, background job queue, or GRC ingestion works.

The test exposed missing `kind`/`apiVersion` on Kubernetes list items. The scanner now restores these fields from the typed API collection. This fix is deployed.

The production portal acceptance test also passed on 2026-10-01. The Kubernetes Demo button created namespace `cg-225e97ac6bdf`, queued job `job-935ac1b40b6b46c5a71328cf4ee8cbb7`, and completed scan 20 with 47 findings, including 14 lab-specific findings. The scan artifact was stored in MinIO with a checksum. Automatic cleanup recorded no errors, and an independent VM check confirmed both the namespace and demo pod were gone. The initial failed attempt was cleaned up as well.

The live test exposed a second deployment defect: background workers did not receive `ENCRYPTION_KEY`, so they could not decrypt the portal's saved kubeconfig. The scanner, sandbox, and scheduler workers now receive the existing key; it was not rotated. Six focused regression tests pass. The deployed release is `6342ae2a` on `codex/decoupled-scanner-deployment`.

The GRC/MinIO repair retest also passed on 2026-10-01. Scan 21, job `job-21ff1cf2884c4873a90c7cde8b6`

recorded 47 findings, including 14 for namespace `cg-e51f28060122`. Its scan artifact was stored in MinIO, and automatic cleanup recorded no errors. An independent VM check confirmed the namespace and pod were gone.

The deployed GRC connector service received this fresh lab proof, scan evidence, and lab findings with the correct tenant identity. Its snapshot contained 16 evidence artifacts. The GRC SSO failure was traced to conflicting HTML/Nginx content security policies; both now allow the exact intended Keycloak origin. Discovery CORS passes, and unauthenticated GRC connector requests still return HTTP 401. A full browser login remains pending the user's sign-in; the repair retest was server-side, not a new website-button test.

Run the smoke test using a Python environment with the project's Kubernetes and YAML dependencies:

```
python deploy/kubernetes-lab/smoke-hostinger-lab.py \
  --host <lab-vm-ip> \
  --ssh-key <private-key-path> \
  --kubeconfig <private-kubeconfig-path> \
  --report <report-json-path>
```

The smoke test schedules a server-side five-minute cleanup timer before creating objects and also deletes the namespace in its own cleanup handler. This timer is a smoke-test failsafe, not a replacement for the production sandbox worker's TTL enforcement.

Existing Dokploy Image Cache

On 2026-10-01 the configured MinIO client image could not be pulled from Docker Hub, and the matching pinned official server/client manifests were unavailable on Quay. Dokploy now initializes the evidence bucket with the application's S3 SDK instead of a separate `mc` image.

The cache workaround has now been replaced by `deploy/dokploy/minio.Dockerfile`. It builds the official security release `RELEASE.2025-10-15T17-29-55Z` from immutable source commit `9e49d5e7a648f00e26f2246f4dc28e6b07f8c84a`, using Go 1.25.8. The original data volume and credentials are unchanged. Old `MINIO_IMAGE` and `MINIO_PULL_POLICY` environment entries are no longer used by this compose service.

Before the upgrade, all 44 existing objects were backed up to the private persistent archive `/app/reports/.storage-backups/minio-before-source-build-20261001.zip`. After the upgrade, every object matched its backup checksum; a temporary S3 write/read/delete probe also passed. Keep this backup and the old cached image until the recovery window ends. Do not run `docker compose down -v`.

The production code repair is commit `aa52965c` on `codex/decoupled-scanner-deployment`. Eight focused regression tests pass, and both scanner and GRC compose files validate. The MinIO community source is archived upstream, so the pinned build solves image availability but is not a promise of future upstream security support. Preserve its AGPL license and source attribution; plan a maintained S3-compatible storage migration separately.